

验收流程

2026年5月24日 19:34

第一步：启动环境

1. 启动数据库（Docker）
cd E:\Project\schoool\cat-cafe\cat-cafe
docker compose up -d mysql

这个命令用 Docker 启动 MySQL 8.0，数据库文件在 mysql_data 卷里，关掉也不会丢。

2. 启动后端
cd E:\Project\schoool\cat-cafe\cat-cafe\backend
.venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0

FastAPI 后端跑在 http://127.0.0.1:8000，--reload 表示代码改了自动重启。

3. 启动前端
cd E:\Project\schoool\cat-cafe\cat-cafe\web\cat-cafe-ui
npm run dev

Vite 开发服务器跑在 http://localhost:5173。

老师：同学你好，请介绍一下你在这个项目中负责什么？

老师，您好，我主要是负责后端和服务器的开发

老师：好，那后端用的什么技术栈？给我看看你的项目结构。

你的技术栈（记住这行就够了）

后端 Python FastAPI + 数据库 MySQL 8.0 + 前端 Vue 3 + Element Plus + 部署 Docker + Nginx，跑在阿里云上。

后端用 Python 的 FastAPI 框架，ORM 是 SQLAlchemy，数据库 MySQL 8.0。前端用 Vue 3 加 Element Plus 组件库，Vite 打包。部署用 Docker Compose 编排了四个容器——Nginx

反向代理、Backend、MySQL、Adminer，都跑在阿里云 ECS 上，域名 pixelcat.tech。

要展示的东西：

- 1. HeidiSQL — 点开数据库，6 张表一目了然
- 2. 服务器— 前端跑起来，登录 admin / admin123
- 3. 服务器 文档，41 个 API 全列出来

老师：好，那说说 orders 表的设计。一个顾客一次买了撸猫券和可乐，你的数据库怎么存？

打开 HeidiSQL，点 orders 表，找到 batchNo 列。然后查这条：

```
SELECT orderId, batchNo, productId, productQuantity, totalAmount, orderStatus  
FROM orders WHERE batchNo = 'batch_007';
```

看到结果了吗？同一批 batch_007 分成了 3 条记录，分别对应撸猫券、可乐、猫条零食。

我们用 batchNo 批次号策略。每行订单只存一个商品，同一批下单的多行共享一个 batchNo。查询时按 batchNo 聚合展示，取消或支付时按 batchNo 批量更新。单商品下单 batchNo 为空，完全兼容。

老师：前端你怎么做的？给我看看路由和权限怎么控制的。

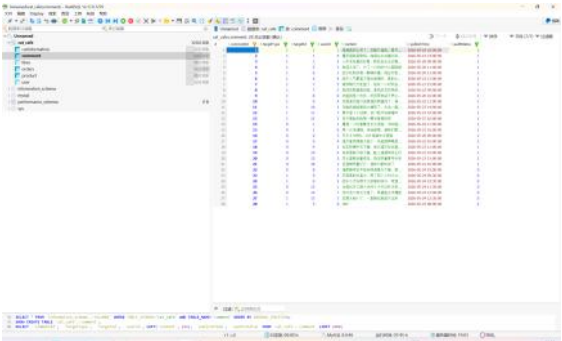
打开 http://localhost:5173，分别演示两个登录：

- 1. 登录 admin / admin123 → 进管理端（左侧有菜单栏）
- 2. 退出，登录 test / 123456 → 进顾客端（顶部有导航栏）

第二步：打开工具

- HeidiSQL → 连 127.0.0.1:3306，root / yang，看数据库
- 浏览器 → http://localhost:5173，打开前端
- 浏览器 → http://127.0.0.1:8000/docs，打开 API 文档

来，现在打开 HeidiSQL，把数据库展开，然后告诉我你看到了几张表？



不复杂，一句话就能说完：

"用户下单买东西，买家卖家都能评论点赞，管理员管一切。"

展开 HeidiSQL，看着表名说：

6 张表：user、orders、product、catinformation、comment、likes

关系就两条线：

- user → orders → product（顾客下单买商品）
- user → comment / likes（顾客评论点赞猫和商品）

照着 HeidiSQL 左边目录念就行，老师就是在看你知不知道每张表干嘛的。

老师：项目怎么部署的？说说服务器架构。

打开你的阿里云控制台或者直接画图说：

"Docker Compose 编排了四个容器——Nginx、Backend、MySQL、Adminer。Nginx 是唯一公网入口跑在 80 端口，反向代理 /api/ 到后端 8000，/cats/、/products/ 等静态资源直接由 Nginx 返回，不经过后端。MySQL 只在内网暴露，外部访问不到。Adminer 给老师用的，网页版数据库管理。"

Nginx 管入口 → Backend 管业务 → MySQL 管数据 → Adminer 方便查看
老师追问：图片怎么存的？

"用户上传图片到后端，存到宿主机 ./uploads/ 目录。这个目录同时挂载给了 Nginx——所以 Nginx 能直接读。用户访问 /cats/xxx.jpg 时，Nginx 直接返回文件，不经过 Python，性能最好。"

浏览器 → Nginx(:80) → /api/* → Backend(:8000) → MySQL(:3306)
→ /cats/* → 直接返回文件

老师最后可能问的三个问题，提前准备答案：

Q1：你觉得项目还有什么可以改进的？

"前端可以加路由守卫，401 全局处理，TypeScript，数据库可以考虑

- 1. 登录 admin / admin123 → 进管理端（左侧有菜单栏）
- 2. 退出，登录 test / 123456 → 进顾客端（顶部有导航栏）

"前端 Vue 3 + Element Plus，路由把顾客和管理拆成两个布局壳。/home/* 是顾客端， /admin/* 是管理端。权限靠后端 JWT 鉴权——登录后 token 存 localStorage，每次请求 axios 拦截器自动把 token 带上。后端根据 userType 判断是管理员还是顾客，没权限返回 403。"

然后加一句：
"前端没有路由守卫，敏感操作靠后端鉴权。即使有人在地址栏输 /admin 绕进来，API 层也会拒绝——安全策略在服务端不依赖客户端。"

Q1：你觉得项目还有什么可以改进的？

|"前端可以加路由守卫、401 全局处理、TypeScript；数据库可以考虑软删除；订单可以考虑引入真实的微信支付接口。"

Q2：遇到最大的困难是什么？

|"订单的多商品策略设计。一开始考虑 order + order_items 双表，后来发现 batchNo 单表方案更简洁，用批次号天然聚合。"

Q3：怎么做测试的？

|"后端 41 个接口全部在 /docs 上手动测试过，覆盖了正常流程和边界——空参数、越界值、权限越权、重复请求都测了。"